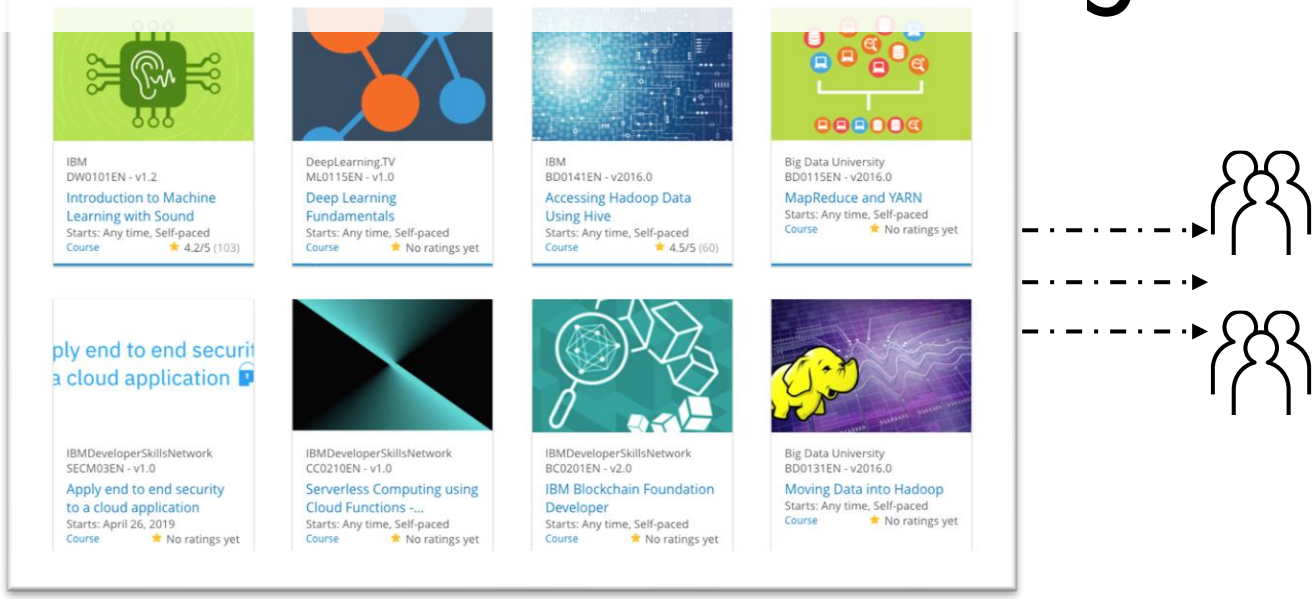
Build a Personalized Online Course Recommender System with Machine Learning

Omar Essam
October 24, 2026



Outline

- Introduction and Background
- Exploratory Data Analysis
- Content-based Recommender System using Unsupervised Learning
- Collaborative-filtering based Recommender System using Supervised learning
- Conclusion
- Appendix

Introduction

Project background and context

- An online learning platform (IBM Skills Network) offers 307 courses in data science, AI, cloud, databases and software development.
- Learners struggle to find the next relevant course in a large catalogue; a personalised recommender surfaces unseen courses that match each learner's interests and raises engagement and completion.
- Data: course titles, descriptions and 14 genre tags, plus 233,306 enrollments with ratings from 33,901 learners.

Problem statement and hypotheses

Problem: recommend new, relevant courses to every learner, and find out which family of models - content-based (unsupervised) or collaborative filtering (supervised) - does it best.

H1: enrollments concentrate on a few popular, introductory courses and genres.

H2: content-based methods can keep the list of new courses per learner under 20 while still reaching most learners.

H3: latent-factor models (NMF, neural embeddings) predict ratings more accurately (lower RMSE) than KNN and a global-mean baseline.

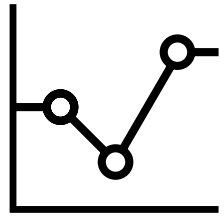
307
courses

14
course genres

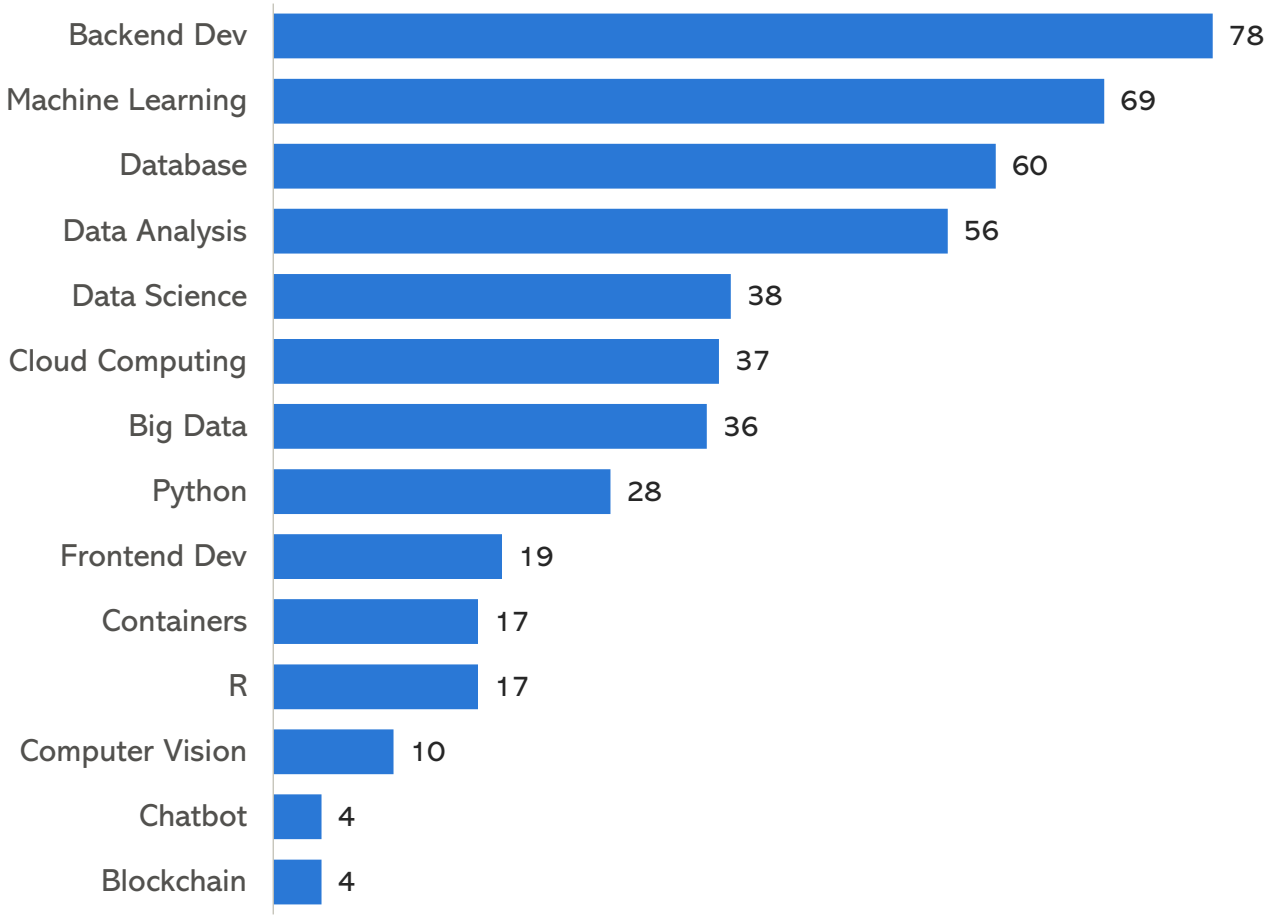
33,901
learners

233,306
enrollments (ratings)

Exploratory Data Analysis



Course counts per genre

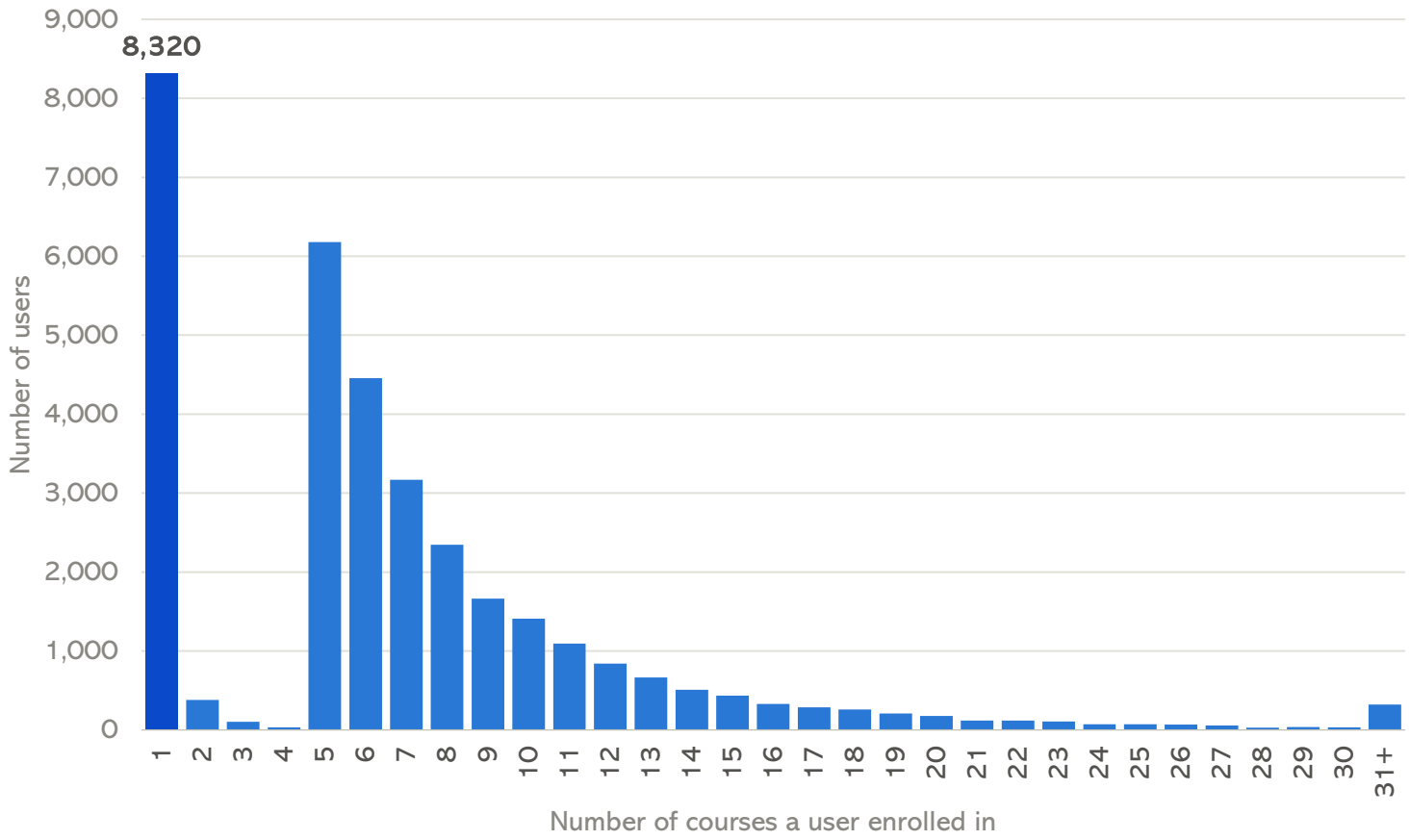


78

courses are tagged Backend Dev, the largest genre (25% of the 307 courses)

- Backend Dev (78 courses) and Machine Learning (69) lead the catalogue, followed by Database (60) and Data Analysis.
- Chatbot and Blockchain are niche, with 4 courses each.
- A course can carry several genres: 473 tags over 307 courses (1.54 per course on average).

Course enrollment distribution

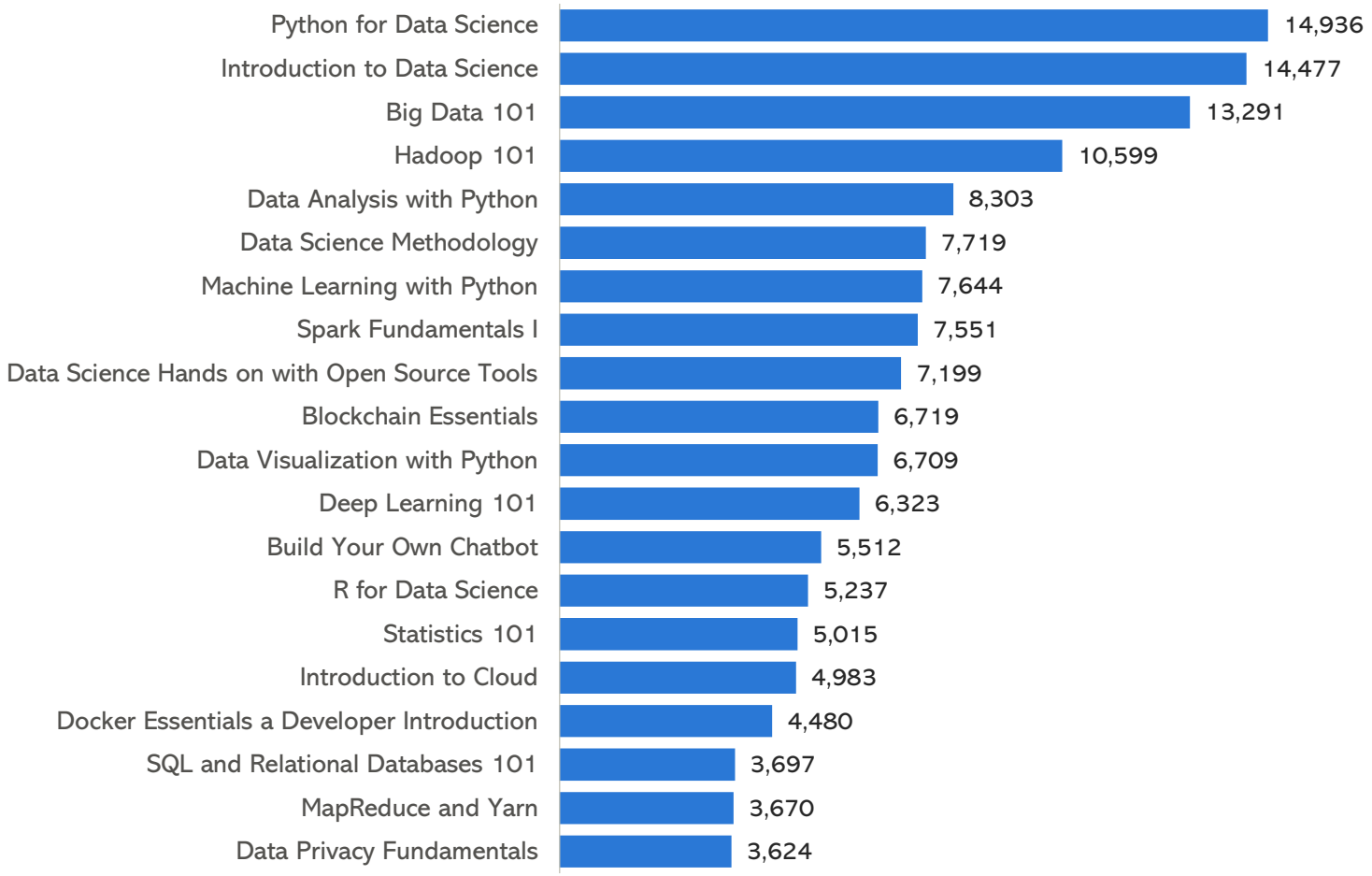


24.5%

of learners (8,320) enrolled in just one course

- Mean 6.9, median 6, max 61 courses per learner (std 5.8).
- The distribution is bimodal: very few learners have 2-4 courses (519), then a second peak at 5 courses (6,179) with a long tail.
- One-course learners are a cold-start group: popularity or content-based recommendations suit them better than collaborative filtering.

20 most popular courses



63.3%

of all 233,306 enrollments go to these 20 of 126 rated courses

- Python for Data Science (14,936), Introduction to Data Science and Big Data 101 lead the list.
- Almost all are introductory courses (101, Introduction, Fundamentals): learners start with foundations.
- This popularity bias (H1 confirmed) means a recommender must work to surface less popular courses.

Word cloud of course titles



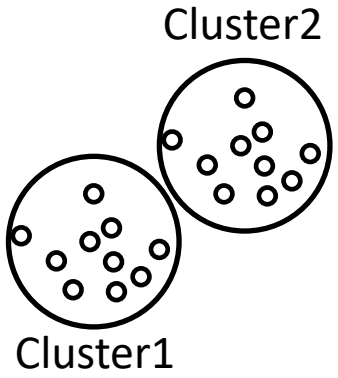
Most frequent title words

data (86), learning (31), science (30), cloud (29), python (27), analysis (27) - counted as the number of titles that contain the word.

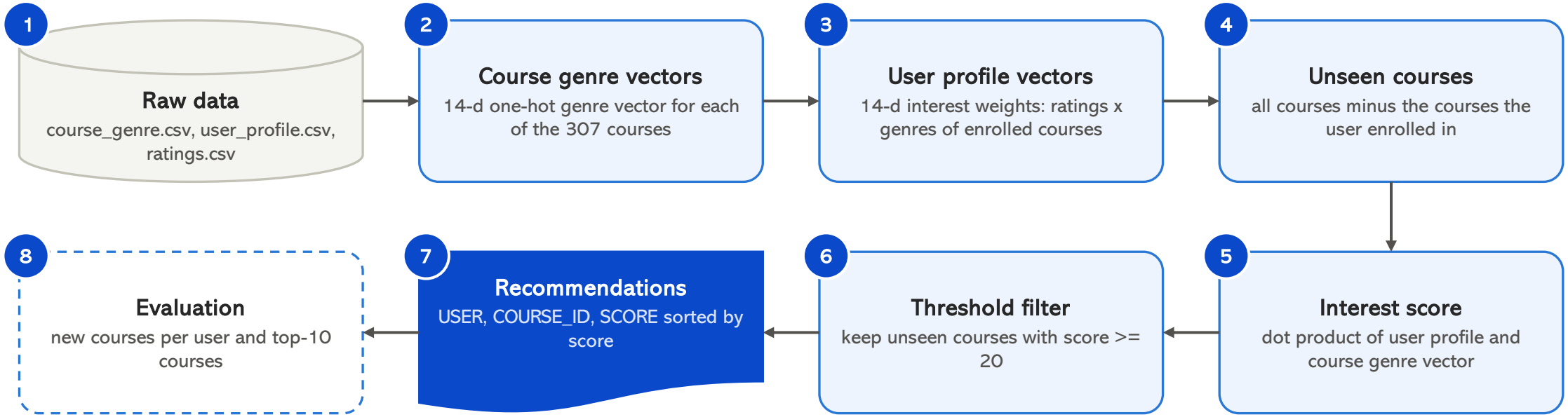
What it tells us

- The catalogue is built around in-demand IT skills: data science, Python, machine learning, cloud, big data, SQL and containers.
- Title words overlap strongly between courses, so text features (bag of words) are a good basis for course similarity.

Content-based Recommender System using Unsupervised Learning



Flowchart of content-based recommender system using user profile and course genres



- Course genre vectors (307 x 14) and user profile vectors (33,901 x 14) live in the same genre space, so a dot product measures how well a course matches a learner's interests.
- Only unseen courses are scored; the score threshold controls list length - the lab default of 10 gave 44 courses per user, so it was raised to 20.
- Evaluation follows the lab questions: average number of new courses per test user and the 10 most frequently recommended courses.

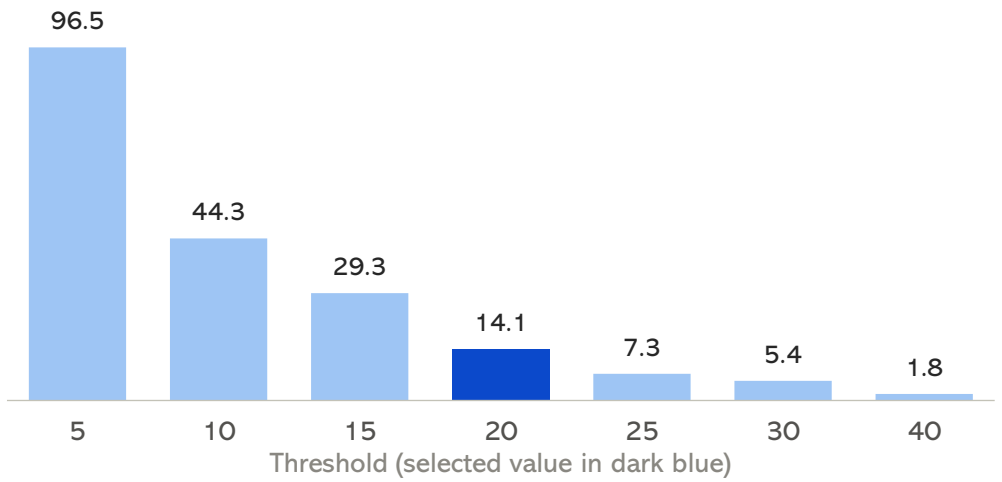
Evaluation results of user profile-based recommender system

Hyper-parameters: recommendation score threshold = 20 (dot product of user profile and course genres); thresholds 5-40 tested, chosen to keep lists under 20 courses.

14.1

new/unseen courses recommended per test user on average (16,554 of 33,901 users receive at least one)

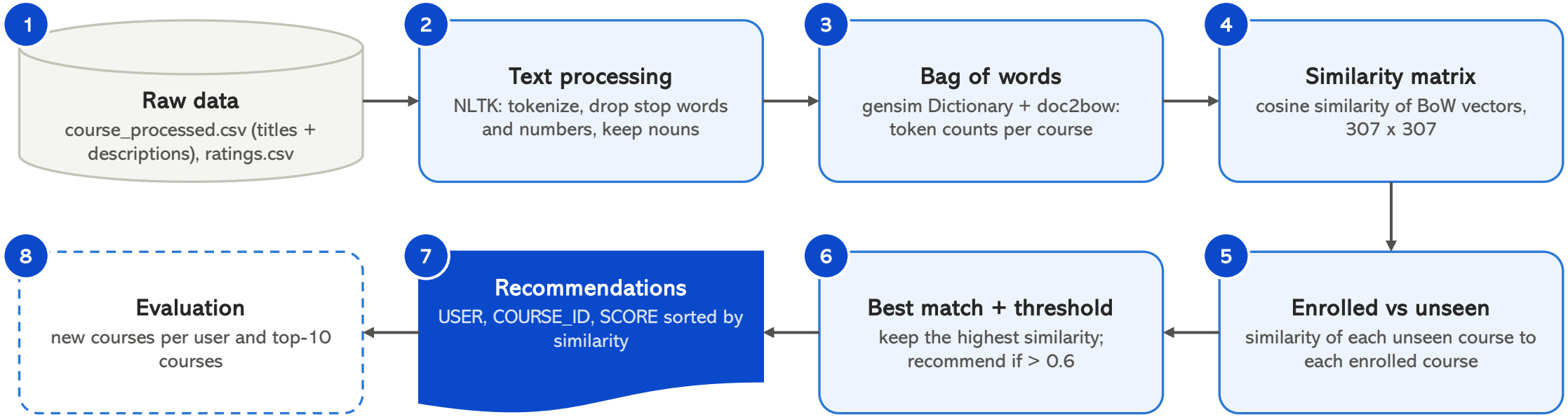
Average new courses per user by score threshold



Top-10 most frequently recommended courses

#	Course	Times
1	Foundations for Big Data Analysis with SQL	9,138
2	Analyzing Big Data with SQL	9,138
3	Getting Started with the Data Apache Spark Makers Build	8,954
4	Analyzing Big Data in R using Apache Spark	8,769
5	Spark Overview for Scala Analytics	7,970
6	Cloud Computing Applications Part 2 Big Data and Applications in the Cloud	7,853
7	Applied Machine Learning in Python	7,671
8	Introduction to Data Science in Python	7,671
9	Accelerating Deep Learning with GPU	7,633
10	Spark Fundamentals II	7,203

Flowchart of content-based recommender system using course similarity



- Each course is represented by the bag of words of its title and description (stop words removed, nouns kept), giving sparse token-count vectors.
- Cosine similarity between BoW vectors measures content overlap; e.g. Machine Learning with Python vs Machine Learning with R = 0.66.
- For every test user an unseen course is recommended when its best similarity to one of the user's enrolled courses exceeds 0.6.

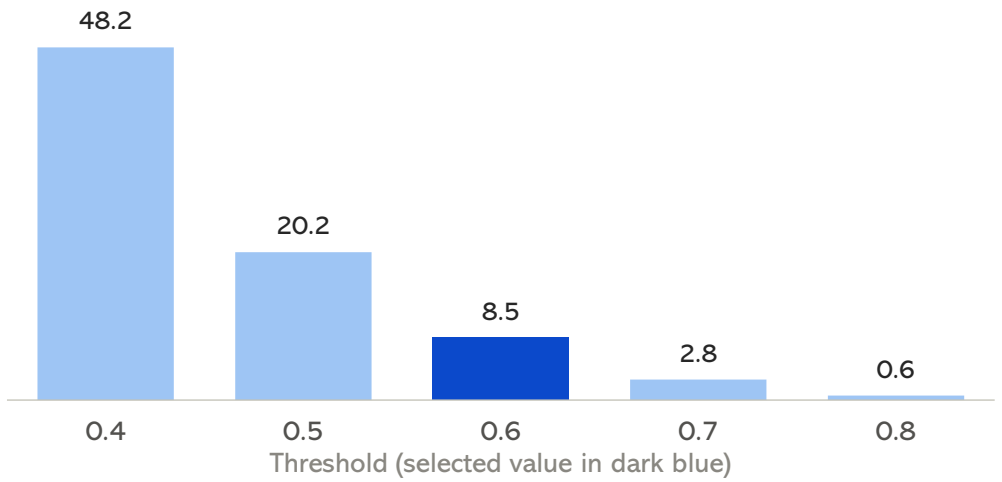
Evaluation results of course similarity based recommender system

Hyper-parameters: cosine similarity threshold = 0.6 on the BoW course similarity matrix (an unseen course must exceed it for at least one enrolled course); 0.4-0.8 tested.

8.5

new/unseen courses recommended per test user on average (29,661 of 33,901 users receive at least one)

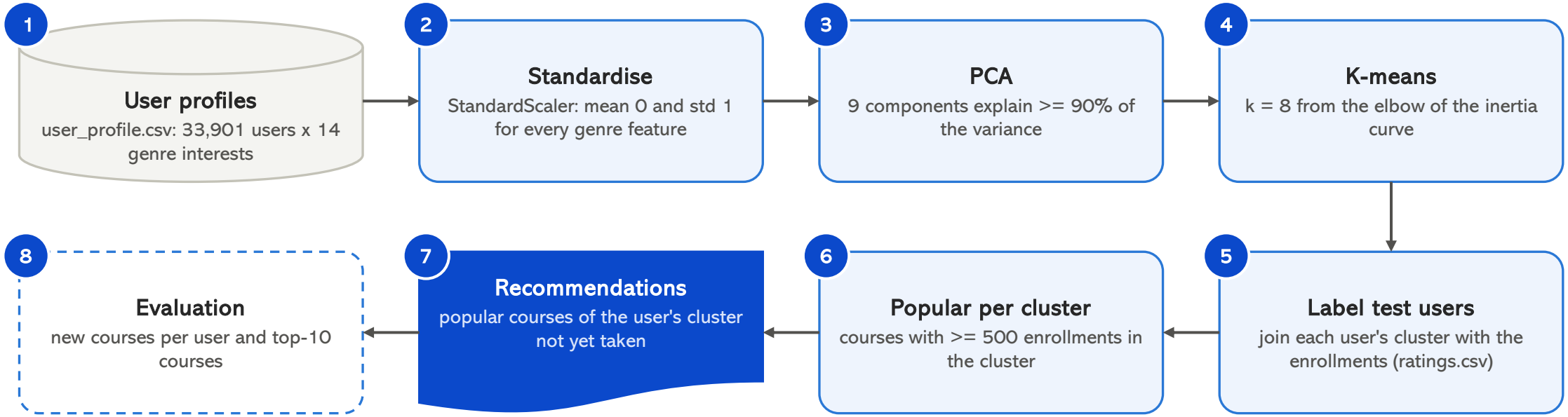
Average new courses per user by similarity threshold



Top-10 most frequently recommended courses

#	Course	Times
1	Data Science with Open Data	15,003
2	Introduction to Data Science in Python (excourse22)	14,937
3	Introduction to Data Science in Python (excourse62)	14,937
4	A Crash Course in Data Science	14,641
5	Data Science Fundamentals for Data Analysts	14,641
6	Big Data Modeling and Management Systems	13,551
7	Foundations for Big Data Analysis with SQL	13,512
8	Fundamentals of Big Data	13,291
9	Introduction to Big Data	13,291
10	SQL Access for Hadoop	12,497

Flowchart of clustering-based recommender system



- K-means was first run on the 14 standardised features (elbow at k = 8), then on 9 PCA components that keep 90% of the variance.
- Each cluster is a learning community; courses enrolled by many members of the community are its popular courses.
- Popularity threshold = 500 enrollments within the cluster; the user's own courses are excluded from the recommendations.

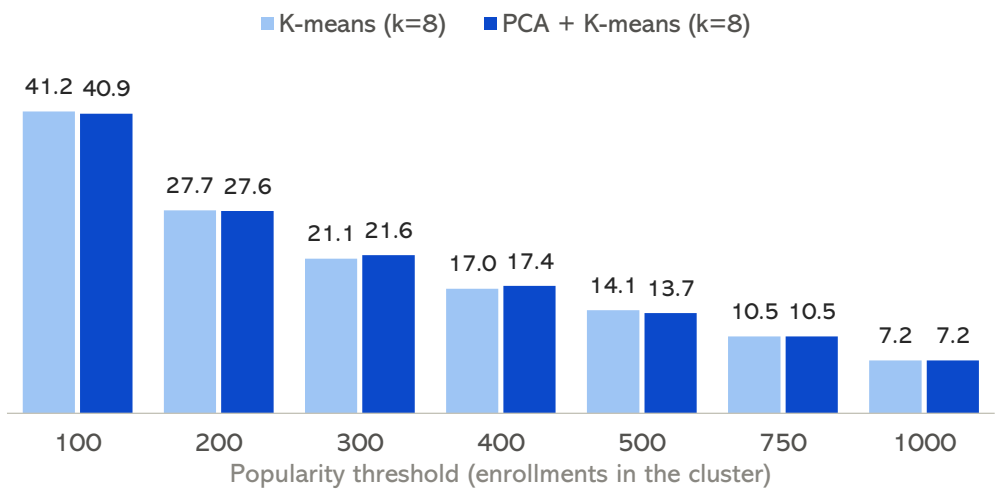
Evaluation results of clustering-based recommender system

Hyper-parameters: PCA n_components = 9 (>= 90% variance), K-means k = 8 (elbow), popularity threshold = 500 enrollments per cluster; 100-1,000 tested, with and without PCA.

13.7

new/unseen courses recommended per test user on average (33,381 of 33,901 users receive at least one)

Average new courses per user, with and without PCA



Top-10 most frequently recommended courses

#	Course	Times
1	Blockchain Essentials	26,721
2	Data Science Hands on with Open Source Tools	23,038
3	Introduction to Cloud	22,828
4	Hadoop 101	22,824
5	Deep Learning 101	22,653
6	Data Science Methodology	22,569
7	Statistics 101	20,691
8	Big Data 101	20,139
9	Introduction to Data Science	18,920
10	Python for Data Science	18,490

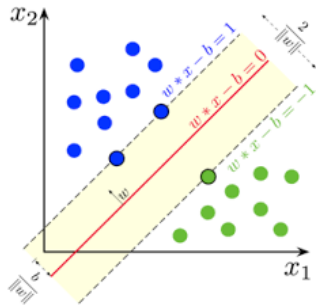
Comparing the three content-based recommender systems

Metric	User profile + genres	Course similarity (BoW)	User clustering (PCA + K-means)
Avg new courses per user	14.1	8.5	13.7
Learners reached (at least one course)	16,554 (49%)	29,661 (87%)	33,381 (98%)
Catalogue coverage (distinct courses)	280 of 307 (91%)	63 of 307 (21%)	50 of 307 (16%)
Total recommendations	479,131	289,738	466,094

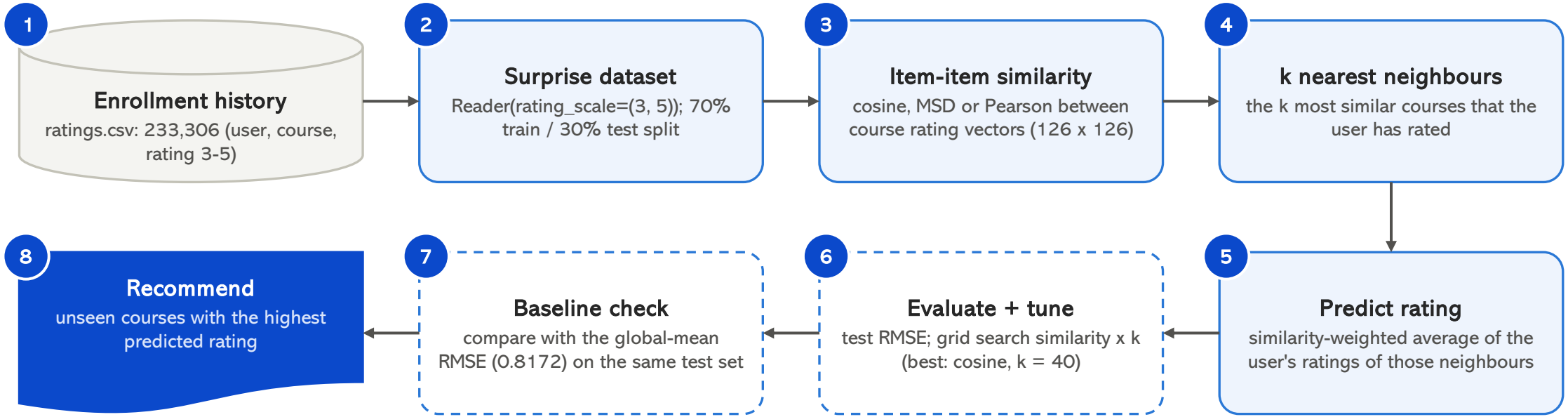
Innovative insight: the three content-based recommenders behave very differently

- User clustering (PCA + K-means) reaches the most learners, and User profile + genres spreads recommendations over the widest part of the catalogue.
- Clustering recommends what is already popular in a learner's group, so it reinforces the popularity bias seen in the EDA; similarity and profile methods expose more long-tail courses.
- Only 2 courses appear in more than one top-10 list, so the methods are complementary: a hybrid (e.g. profile score re-ranked by similarity) is a natural next step.

Collaborative-filtering Recommender System using Supervised Learning

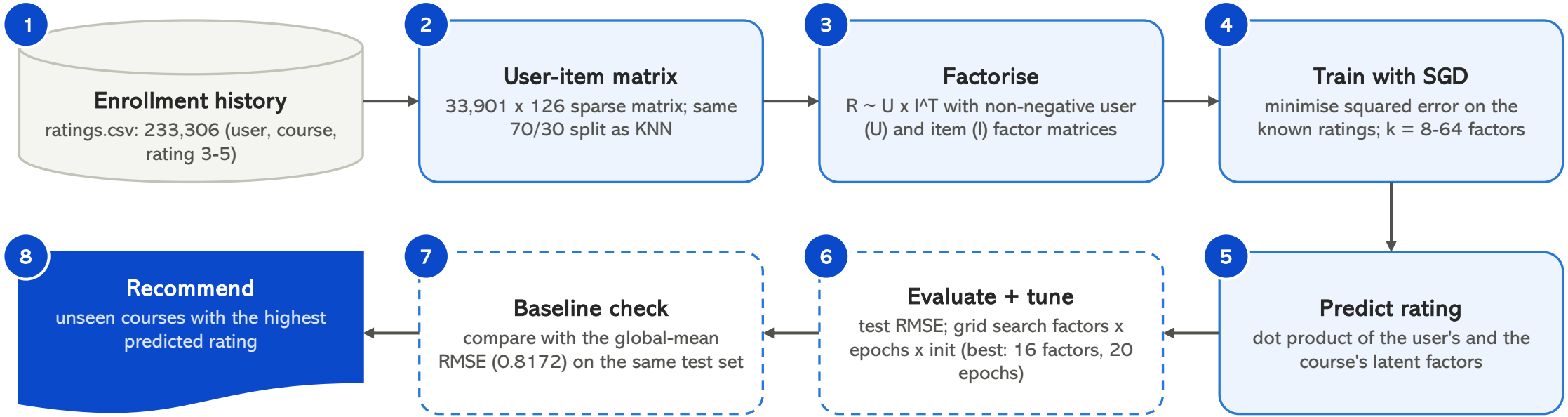


Flowchart of KNN based recommender system



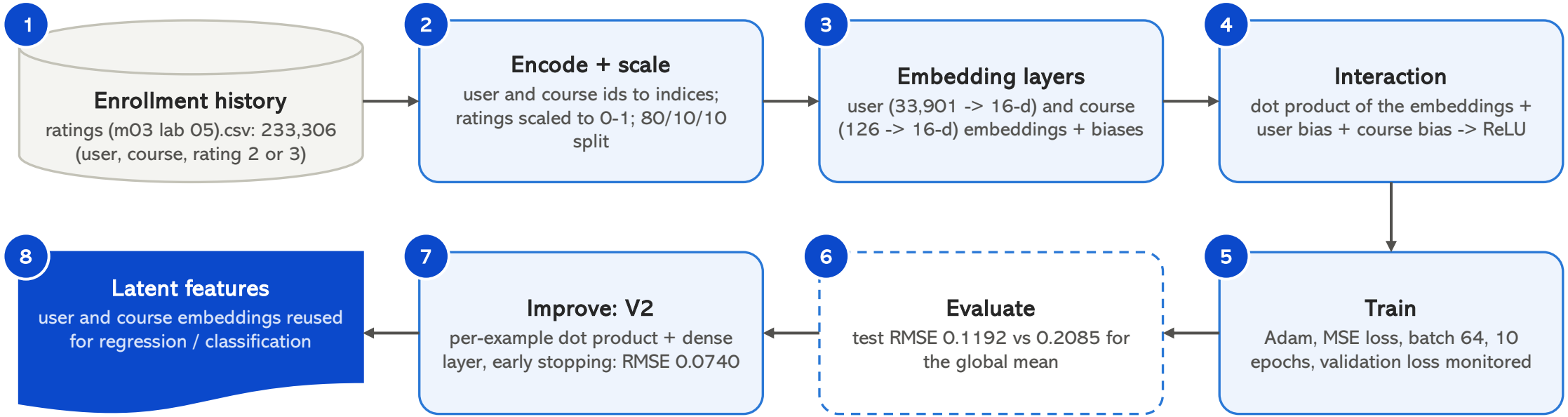
- Item-based KNN: the course-course similarity matrix is tiny (126 x 126), while a user-based model would need a 33,901 x 33,901 matrix.
- Prediction: r(u, i) = sum of sim(i, j) x r(u, j) / sum of sim(i, j) over the k most similar courses j rated by u; grid search over cosine / MSD / Pearson and k = 10-80.
- Best: cosine, k = 40, test RMSE 0.8955 vs 0.8172 for the global mean (ratings.csv carries no signal - see findings). Lab fix: Reader scale (3, 5) instead of (2, 3).

Flowchart of NMF based recommender system



- NMF compresses the 33,901 x 126 rating matrix into k latent factors per user and per course, so no large similarity matrix has to be kept in memory; factors are learned by SGD on the observed ratings only.
- The lab's hint (init_high = 5) starts the factor dot products far above 5, so every prediction is clipped to 5 (RMSE 1.2806); the default init range (0, 1) was added to the grid.
- Best: 16 factors, 20 epochs, init (0, 1) - test RMSE 0.9505 vs 0.8172 for the global mean, on the same test ratings as KNN.

Flowchart of Neural Network Embedding based recommender system



- The one-hot user and course inputs are mapped to dense 16-d embedding vectors - the neural counterpart of the NMF factor matrices.
- Finding: the lab's tf.tensordot(user, item, 2) sums the dot products over the whole batch. RecommenderNetV2 (per-example dot product + dense layer) cuts the test RMSE to 0.0740.
- The embedding matrices (33,901 x 16 and 126 x 16) are the extracted latent features used by the regression and classification labs.

Embedding features: regression and classification results

Regression: predict the rating score

Model	Best hyper-parameters	Test RMSE
Lasso	alpha: 0.001	0.8138
ElasticNet	alpha: 0.001, l1_ratio: 0.5	0.8138
Global-mean baseline	-	0.8138
Ridge	alpha: 100	0.8138
LinearRegression	-	0.8138
HistGradientBoosting	learning_rate: 0.05, max_depth: 3	0.8138

Classification: predict the rating class (3 / 4 / 5)

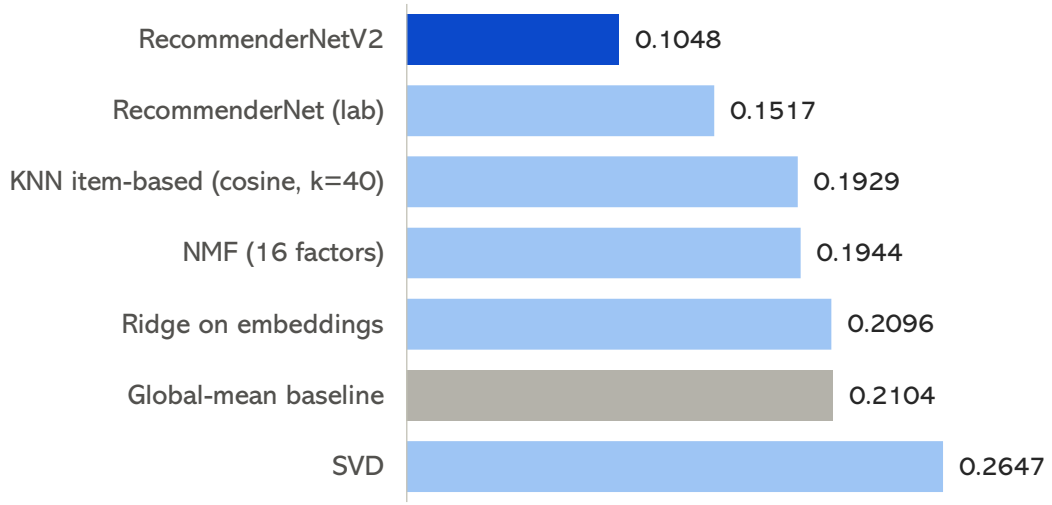
Model	Accuracy	Macro F1	AUC (OvR)
Bagging (trees)	0.336	0.332	0.500
Logistic Regression	0.335	0.327	0.500
Linear SVM	0.335	0.327	-
Gradient Boosting	0.333	0.326	0.499
Random Forest	0.335	0.324	0.503
Decision Tree	0.330	0.215	0.500
Majority-class baseline	0.329	0.165	0.500

- Input X = user embedding + course embedding (element-wise sum, 16 features) for every rating of ratings.csv; label y = rating score / class.
- Best regressor: Lasso (RMSE 0.8138 vs 0.8138 for the global mean); best classifier: Bagging (trees) (accuracy 0.336 vs 0.329 for the majority class, macro F1 0.332).
- Every regressor matches the global mean and every classifier sits at chance (AUC about 0.50): the 3/4/5 ratings of ratings.csv are random, so there is no signal to learn (see findings).

Compare the performance of collaborative-filtering models

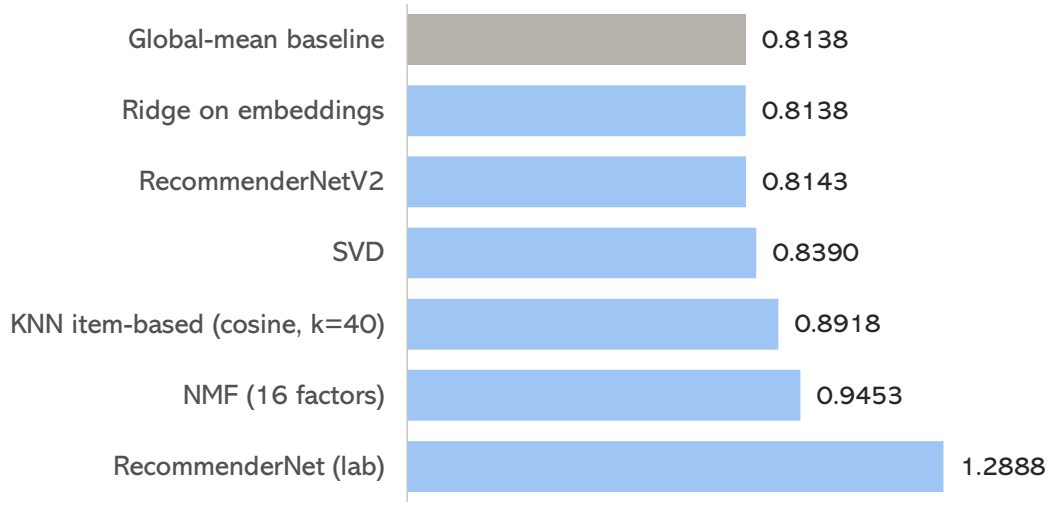
Lab-05 ratings (2/3): the file with real signal

test RMSE, same 70/30 split for every model, rating units, lower is better



ratings.csv (3/4/5): random ratings

test RMSE, same 70/30 split for every model, rating units, lower is better



- **Signal file:** RecommenderNetV2 is best (0.1048, 50% below the global mean 0.2104), then the lab RecommenderNet (0.1517); KNN and NMF improve 8% and 8%; untuned SVD (100 factors) overfits.
- **ratings.csv:** the ratings are random, so the global mean (0.8138) is the best possible score; models that fit noise score worse. Lab runs agree: KNN 0.8955 and NMF 0.9505 vs baseline 0.8172.
- **Metric:** RMSE = square root of the mean squared difference between predicted and true ratings on held-out data - the evaluation metric of every collaborative-filtering lab.

Conclusions

Key findings

- H1 confirmed: the top-20 courses take 63% of all enrollments, and 25% of learners took a single course.
- H2 confirmed: after tuning, the profile, similarity and clustering recommenders suggest 14.1, 8.5 and 13.7 new courses per learner, reaching 49%, 87% and 98% of learners.
- H3 confirmed where the data has signal: RecommenderNetV2 cuts RMSE by 50% vs the global mean and beats KNN and NMF. On ratings.csv, whose ratings are random, no model beats the mean (0.814).
- Data checks and tuning matter as much as the model: default thresholds gave 41-44 courses per learner, and a randomised rating file makes every model look broken.

Recommendations

- Serve new and one-course learners with content-based recommendations (profile or similarity): they need no rating history.
- Use the neural embedding model for learners with history, with item-based KNN as a simple, explainable fallback.
- Blend both in a hybrid ranker, cap lists with a per-user top-N (10-20), deduplicate course versions and add diversity for long-tail courses.
- Next: ranking metrics (precision@k, recall@k) on a time-based split, then an A/B test of click-through and completion.

Bottom line: start every learner with content-based recommendations, switch to the neural embedding model once they have a rating history, and judge every model against a baseline on validated data.

Appendix

Lab notebooks

11 labs solved and executed: m01/06, 07, 10 (EDA, bag of words, similarity); m02/03, 05, 06 (profile, similarity, clustering); m03/03-07 (KNN, NMF, neural network, regression, classification).

Benchmark notebook

m03/08 - Extra - Benchmark of Collaborative Filtering Models.ipynb retrains KNN, NMF, SVD, both neural networks and Ridge on one identical 70/30 split of each rating file.

Results

results/*.csv - the small summaries each notebook exports (threshold sweeps, top-10 lists, RMSE tables). Every number in this deck is read from them.

Report source

m05/Final Project Submission/source: build_report.py fills the course template from the results; export_pdf.ps1 saves this PDF with PowerPoint.

Environment

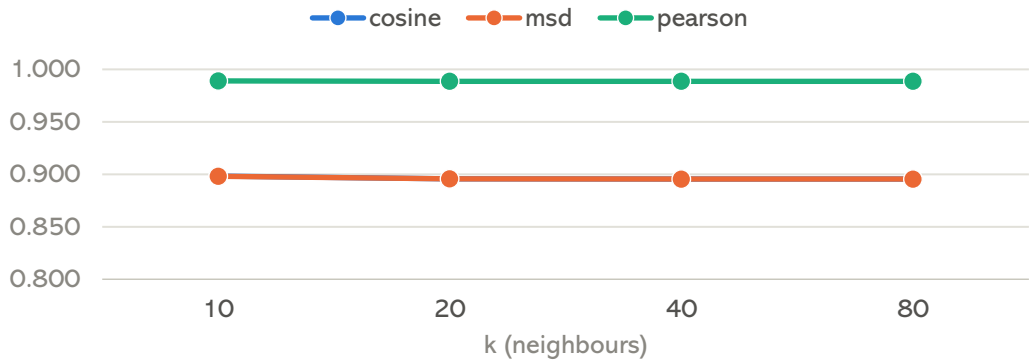
Python 3.11, pandas 2.2, scikit-learn 1.5, scikit-surprise 1.1.4, TensorFlow 2.17 (Keras 3), gensim 4.3, NLTK 3.9, wordcloud 1.9, python-pptx 1.0.

Key code: profile scoring

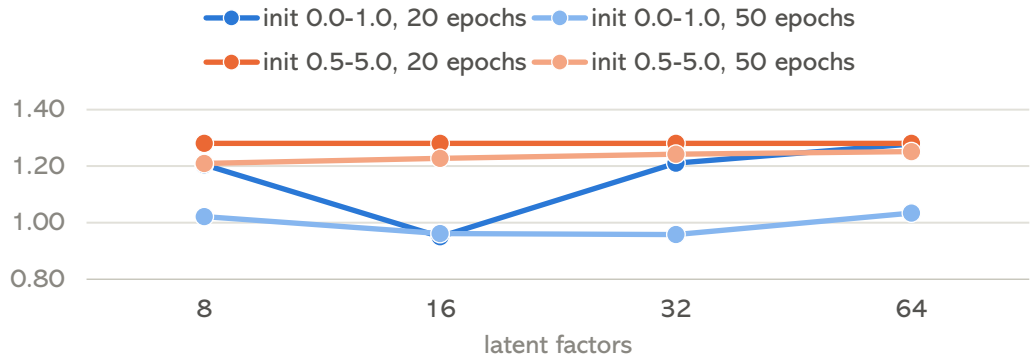
```
# interest score of every course
# for every learner, vectorised
scores = profiles @ genres.T
scores = scores.mask(enrolled)
recs = scores >= 20
recs.sum(axis=1).mean()  # 14.1
```

Appendix: hyper-parameter tuning

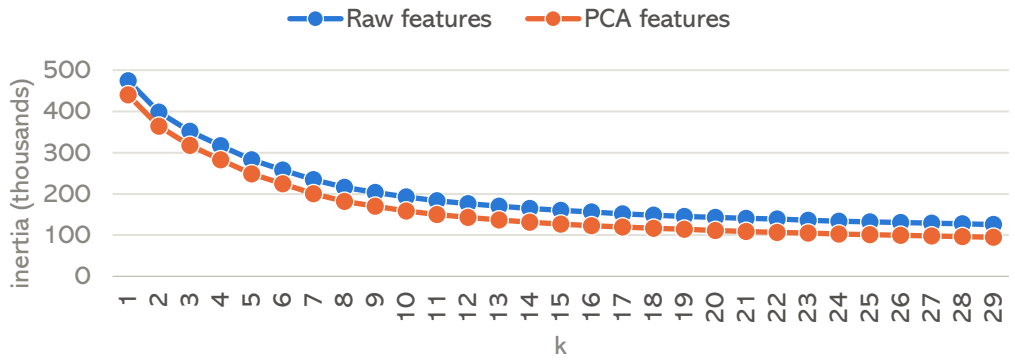
KNN: test RMSE by similarity and k (cosine and MSD overlap)



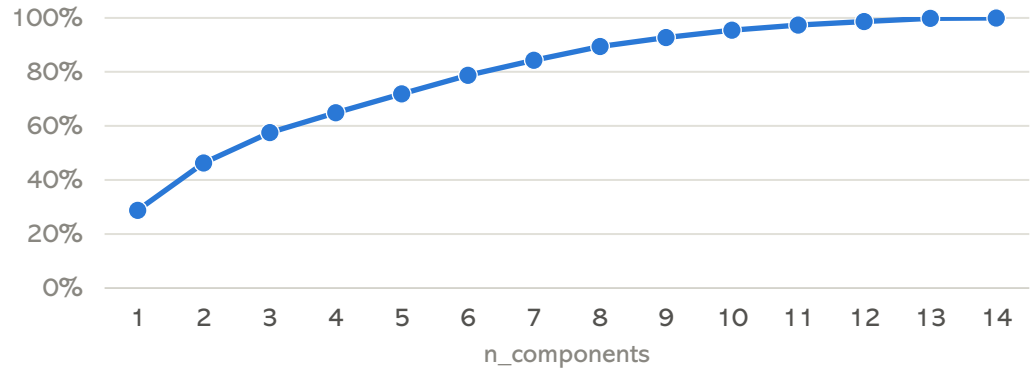
NMF: test RMSE by factors, epochs and initialisation range



K-means elbow (k = 8 raw, 8 on PCA)



PCA: accumulated explained variance (9 components >= 90%)



Appendix: data-quality findings

ratings.csv has random ratings

It holds the same 233,306 (user, course) pairs as the lab-05 file, but its ratings are spread evenly (3: 33.4%, 4: 33.4%, 5: 33.2%) and uncorrelated with the lab-05 ratings (r = -0.003). No model can beat the global mean on it; the benchmark retrains every model on both files.

Rating scale in the KNN / NMF labs

The labs read ratings.csv with Reader(rating_scale=(2, 3)); Surprise clips predictions to that range, so every estimate would be 3. Fixed to (3, 5).

NMF initialisation in the lab hint

init_low = 0.5, init_high = 5 makes the initial factor dot products far larger than 5, so after 20 epochs every prediction is clipped to 5 (RMSE 1.2806). The default range (0, 1) was added to the grid search.

Batch-wide dot product in RecommenderNet

tf.tensordot(user_vector, item_vector, 2) contracts both axes, so every example in a batch gets the same interaction term. RecommenderNetV2 (per-example dot product + dense layer): test RMSE 0.0740 vs 0.1192.

Default thresholds give long lists

Score threshold 10 recommended 44 courses per learner and popularity threshold 100 about 41; every threshold was swept and set to stay under the 20-course guideline.

Duplicate course versions

9 course pairs have identical text (similarity 1.0), e.g. Deep Learning with TensorFlow (ML0120EN / ML0120ENv2). They take slots in the top-10 lists and can recommend a course the learner already took under another id.